

BLM101 3.proje

Ali Uçar
24360859022

Konu başlıkları

1.Bilgisayar mimarisi temelleri.

- CPU,ana bellek,veri yolu
- Depolanmış program konsepti
- Makine dili
- RISC ve CISC mimarileri

2.Temel mantık kapıları ve doğruluk tabloları.

- AND,OR,XOR,NOT

1.Bilgisayar mimarisi temelleri

- **CPU(Central Processing Unit/Merkezi İşlem Ünitesi)**

CPU bilgisayarın temel işlem birimidir ve günümüzde bilgisayarlar,telefonlar,IoT cihazları gibi milyarlarca cihazda bulunur.Amacı bir veriyi istenen şekilde dönüştürmektir.Bu işlevi sağlamak için birden fazla alt birime sahiptir:

1.Aritmetik/Mantık birimi

2.Kontrol birimi

3.Yazmaç

1.Aritmetik/Mantık birimi(ALU):Bu birim çeşitli matematik işlemlerini(toplama,çıkarma vb.) ve mantık işlemlerini(AND,OR vb.) işlemekle görevlidir.

- $10+5=15$
- $20-8=12$
- $1 \text{ XOR } 0 = 1$
- $\text{NOT } 1=0$

2.Kontrol birimi:CPU'nun giriş/çıkış birimleri,ALU ve yazmaçlar arasındaki veri trafiğini denetler.İlk olarak bellekten alınan komutları analiz eder daha sonra gerekli birimlere sinyal gönderir.

- Toplama yap > ALU
- Veri oku > Ana Bellek

3.Yazmaçlar:Yüksek hızlarda çalışan ve geçici olarak veri saklayan belleklerdir.İşlemci hızlı erişim amacıyla bazı verileri RAM,SSD veya harddisk yerine burada depolar ve kullanır.Kullanım amacına göre ikiye ayrılır:

- **Genel amaçlı yazmaç:Veriler veya işlem sonuçları geçici olarak saklanır.**
- **Özel amaçlı yazmaç:Spesifik amaçlara sahip yazmaçlardır.Bunlara örnek olarak:**
 - a.Program sayacı (PC):Programın ilerlemesini saklar.CPU buradaki veriye göre sonraki programdaki kodda sonraki satıra geçer ve işlemi sürdürür.**
 - b.Komut yazmacı(IR):CPU'nun program sayacında geldiği komutu saklar.**

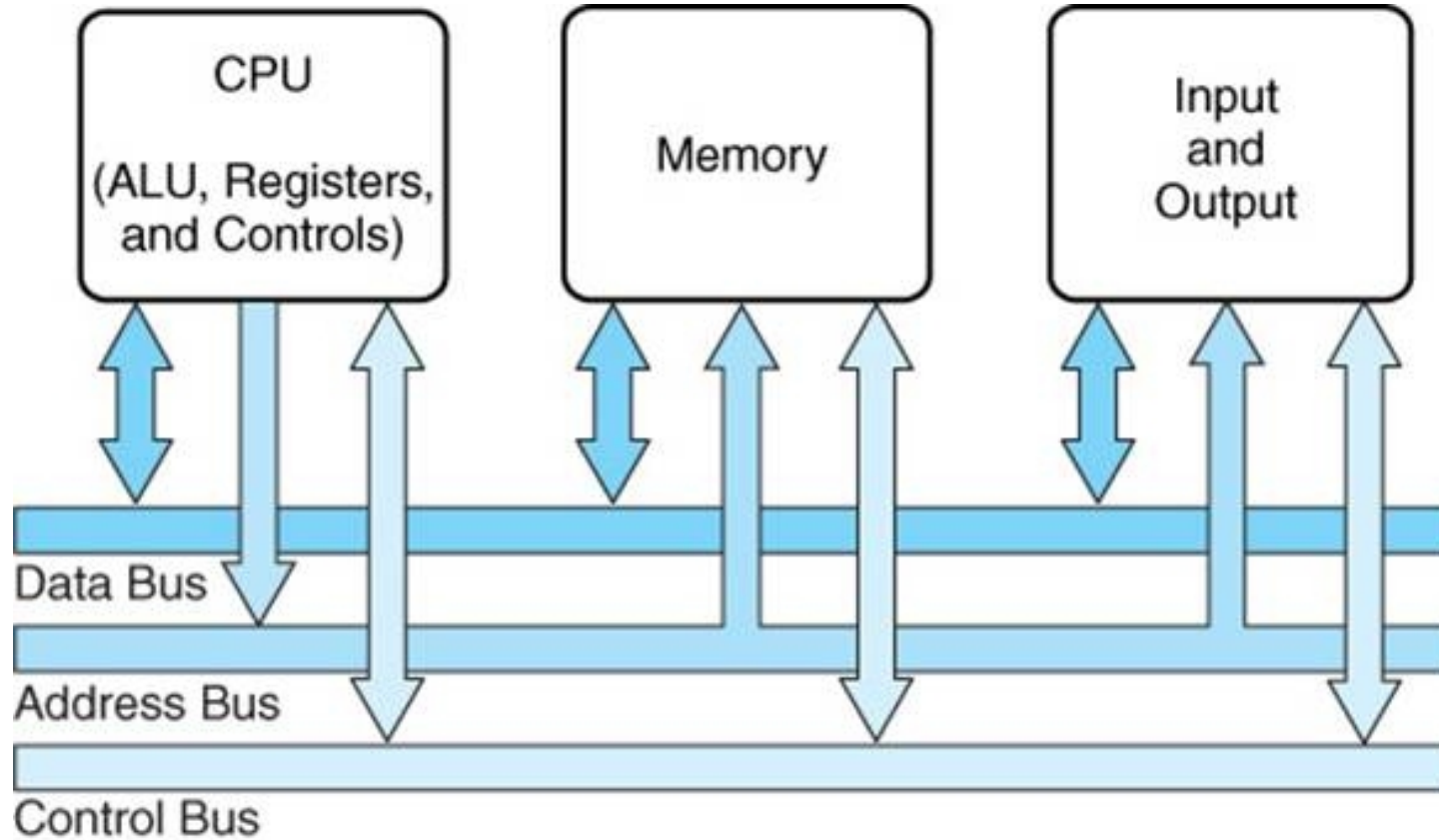
- **Ana bellek(RAM)**

CPU herhangi bir veri olmadan işlem yapamaz.Bu nedenle ana bellek bu verinin depolanması için kullanılır.Ana bellek kalıcı depolama aygıtına (HDD,SSD) göre oldukça hızlıdır ancak içerisindeki veriyi tutması için sürekli elektrik erişimine ihtiyaç duyar,bilgisayar kapandığında veya elektrik kesildiğinde içindeki veri kaybolur.Bu sebeple bilgisayarlar önemli verileri kalıcı depolama aygıtına aktarır ancak OS(İşletim sistemi) ve aktif programlar ana belleği kullanır.

- **Veri yolu(BUS)**

CPU işeyeceği verileri taşımak,yeni verileri almak ve bunların sonuçlarını kaydetmek için bilgisayarın diğer parçaları ile iletişim kurmak zorundadır.Bu amaçla bilgisayarın anakartı üzerinde verileri,adresleri ve kontrol sinyallerini taşıyan yapıya veri yolu adı verilir.Taşınacak veri tiplerine göre birden fazla veri yolu kullanılabilir.CPU veri yolu sayesinde belleğe erişebilir ve üstte belirtildiği gibi verileri adresleri kullanarak ana belleğe yazabilir veya okuyabilir.Ayrıca CPU portlarla ve klavye fare,gibi cihazlara da veri yolu aracılığıyla iletişim kurar.

CPU, Veriyolu, Ana Bellek gösterimi

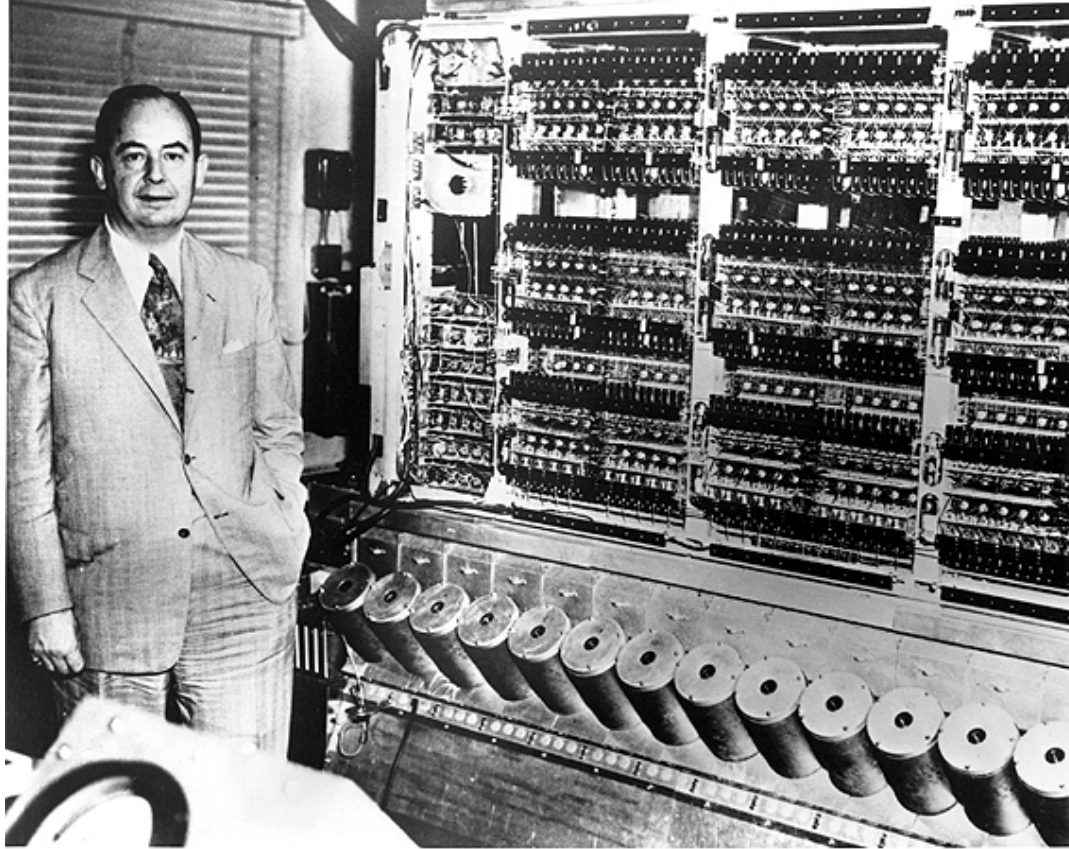


- **Depolanmış program konsepti(Stored program concept):**

Eskiden bilgisayarlar tek bir işleve sahipti ve başka bir işi yerine getirmek için devrelerin yeniden kurulması gerekiyordu. John von Neumann'ın ortaya attığı bu ilke ile bilgisayarların CPU ve ana bellek gibi parçaları sayesinde programlabilmesi ve CPU'daki kontrol biriminin ana bellekteki veriye göre aritmetik birimini yönetmesi dolayısıyla birden fazla işlevi donanımda değişiklik yapmadan yerine getirilmesi sağlandı. Temel çalışma örneği:

- Program sayacı>Ana bellek>Kontrol birimi>ALU>çıktı

John von neumann ve EDVAC bilgisayarı



- **Makine dili**

CPU ile iletişim kurmak için kullanılan dile makine dili denir. Makine dili tamamen ikili sayı(binary) sisteminden yani 1 ve 0 ların kombinasyonlarından oluşur. Assembly ve yüksek seviye programlama dilleri aslında makine dilinin kullanması zor, hatta günümüzdeki programların yazılamayacağı kadar zor olmasından geliştirilmiştir ve bu diller de programa çevrilirken ikili sisteme çevrilmektedir. Makine dili, CPU nun anlayacağı şekildeki makine talimatlarından oluşan ikili kod dizileriyle çalışır ve bu talimatlar her CPU mimarisine göre farklılık gösterebilir. Makine komut yapısı temelde 2 kısma ayrılır:

- **Op-code:**Yapılacak işlemi belirtir.
- **Operand:**İşlemin yapılacağı adresi verir

- **Genelde kullanılan makine dili mimarileri**

1.RISC

- **Daha az sayıda komuta sahiptirler.**
- **Genelde kullanıldıkları cihazlar uzun pil ömrü,düşük güç tüketimi ve sessiz çalışmalarıyla bilinir.**
- **Örn:ARM,Apple bilgisayarları...**

2.CISC

- **Daha fazla sayıda komuta sahiptirler.**
- **Genelde yüksek performans sunarlar ancak çok ısınır ve fazla güç tüketirler.**
- **Örn:Intel,AMD**

- **Makine döngüsü**

CPU çalıştığı süre boyunca sürekli şu üç adımlı döngüyü tekrarlar:

- 1.**Getir(Fetch):**CPU program sayacını kontrol eder ve bu adresteki veriyi ana bellekten ister.Gelen komut CPU içindeki komut yazmacında depolanır ve program sayacı bir arttırılır.
- 2.**Çöz(Decode):**CPU komut yazmacındaki ikili sayı sisteminde yazılmış makine komutunu analiz eder ve gerekli devreyi hazırlar.
- 3.**Çalıştır(Execute):**CPU analiz ettiği makine kodunu gerekli birimde çalıştırır ve çıktıyı bir yazmaca veya ana belleğe yazar.Döngü burada tekrar ilk adıma döner

- **Bazı temel makine komutu tipleri**

- 1.Kontrol komutları**

- Eldeki programın çalıştırılması veya program yazarının kontrolünü gerçekleştiren komutlardır.(Branch,jump)

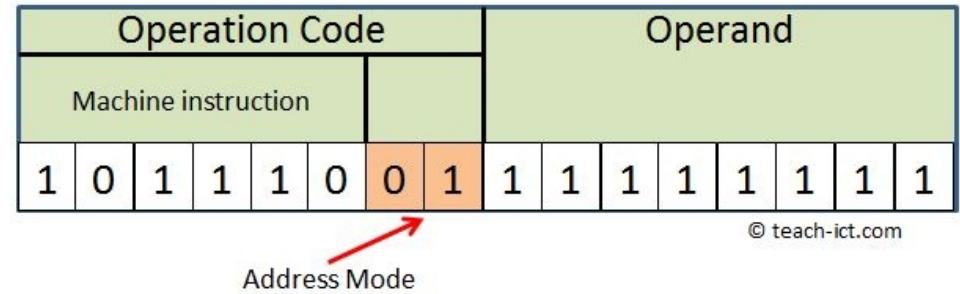
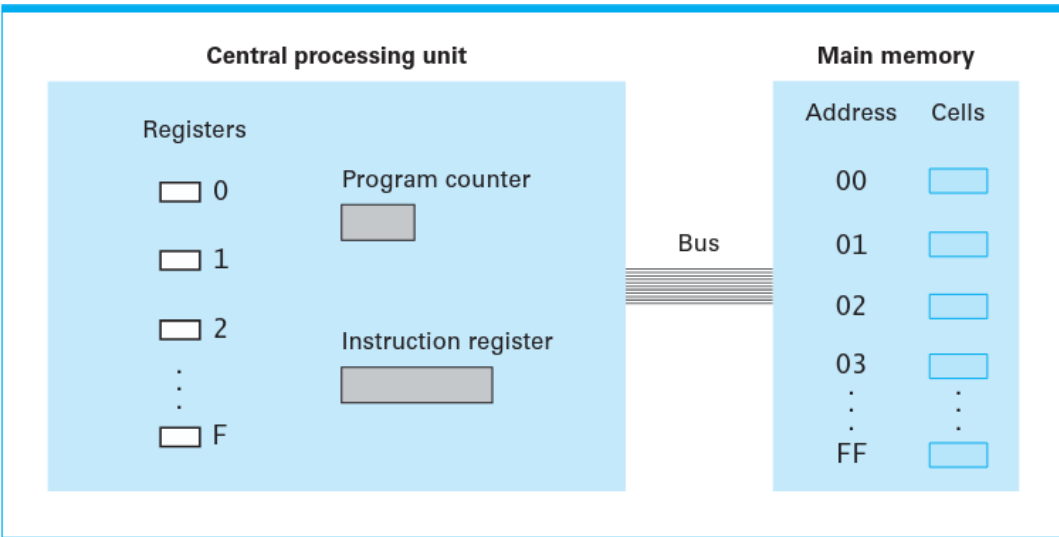
- 2.Aritmetik/Mantık komutları**

- Kontrol biriminin ALU'ya verdiği komutları içerir.(*, +,AND,XOR,SHIFT,ROTATE)

- 3.Ver transferi**

- Veriyi bir konumdan başka bir konuma taşımaya içeren komutlardır. (LOAD,STORE)

- Bilgisayar temel birimleri ve op-code,operand örnek fotoğraf



- Örnek op-code tablosu

Machine Code	Assembly	Description
0000	END	Stops the program
0001	ADD	Adds contents of memory location to accumulator
0010	SUB	Subtracts contents of memory location from accumulator
0011	MULT	Multiplies contents of memory location to accumulator
0100	DIV	Divides contents of memory location into accumulator
0101	LDA	Copy contents of memory locations into the accumulator
0110	STO	Copy contents of the accumulator into the memory location
0111	IN	Input from Input Unit to memory location
1000	OUT	Output contents of memory location to Output Unit
1001	JMP	Transfer Control to Instruction in Named Location
1010	JNZ	Jump if contents of Accumulator is not Zero

- **2+3 işlemi için örnek uygulama:**
- **Adres 1=2 Adres 2 =3**
 - 1.Program sayacı ilgili komuta gelir ve ilk veri adres 1 den yazmaç 1 e yazılır.(LOAD)
 - 2.Program sayacı tekrar artar ve ikinci veri adres 2 den yazmaç 2 ye yazılır.(LOAD)
 - 3.İşlemci op-code u çözer ve ALU'yu aktifleştirerek çıktıyı yazmaç 3 e yazar.(+)
 - 4.Sonuç bellekte adres 3 e yazılır.(STORE)

2.Temel mantık kapıları ve doğruluk tabloları

- **Boolean mantığı**

19.Yüzyılda matematikçi George Bool temel mantıkdaki evet ve hayırı 1 ve 0 ile ifade etme fikrini ortaya attı ve bu yıllar sonra bilgisayarların temeli oldu.Bilgisayarlar da 1'i (5V) evet,0'ı (0.5V) hayır olarak alır.Modern CPU'larda bulunan milyonlarca transistör belli düzenlerde bir araya gelerek mantık kapılarını oluşturur.Bu kapılar 1 veya 2 değer olarak bir çıktı üretir.Dört adet kapı türü vardır:

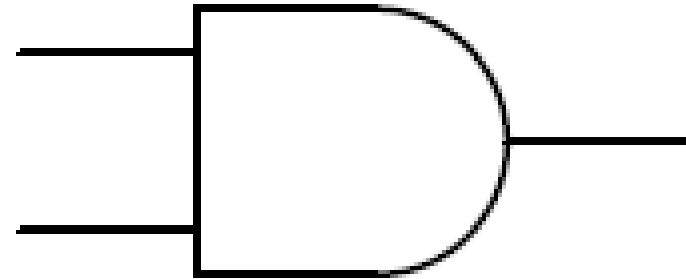
- | | |
|-------|-------|
| 1.AND | 2.OR |
| 3.XOR | 4.NOT |

1.AND kapısı

- AND kapısı verilen değerlerin yalnızca ikisi de 1 ise 1 sonucunu üretir.
- Mükemmeliyetçi bir insana benzetilebilir.
- Ayrıca ALU'da çarpma işlemi için de kullanılır.

AND gate

Input A	Input B	Output
0	0	0
1	0	0
0	1	0
1	1	1



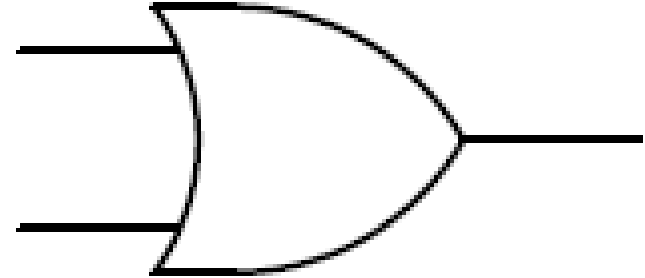
AND

2.OR kapısı

- OR kapısı verilen değerlerin ikisinden biri veya ikisi de 1 ise 1 ,hepsi 0 ise 0 sonucunu üretir.
- Bir odanın kapılarına benzetilebilir.Sadece iki kapı da kapalıysa odaya girilemez.

OR gate

Input A	Input B	Output
0	0	0
1	0	1
0	1	1
1	1	1



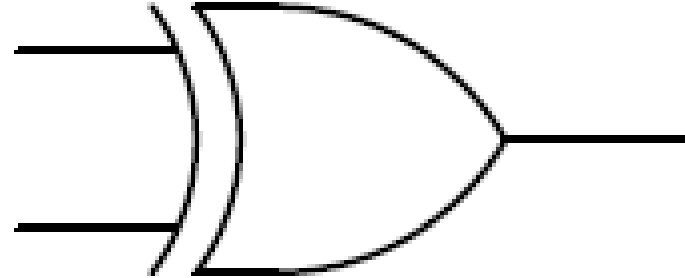
OR

3.XOR kapısı

- XOR kapısı verilen iki değer aynı ise 0 farklı ise 1 sonucunu üretir.
- Manyetik kutuplarına benzetilebilir.Sadece farklı kutuplar birbirini çeker.
- Ayrıca ALU'da toplama işlemi için de kullanılır.

EX-OR gate

Input A	Input B	Output
0	0	0
1	0	1
0	1	1
1	1	0

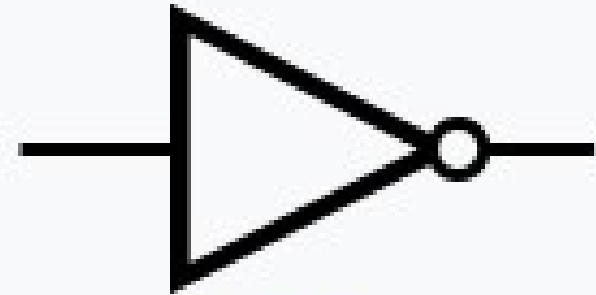


XOR

4.NOT kapısı

- NOT kapısı 1 verildiğinde 0,0 verildiğinde 1 sonucunu üretir,
- Ayrıca ALU'da çıkarma işlemi için de kullanılır.

A (Input)	$Y = \bar{A}$ (Output)
0	1
1	0

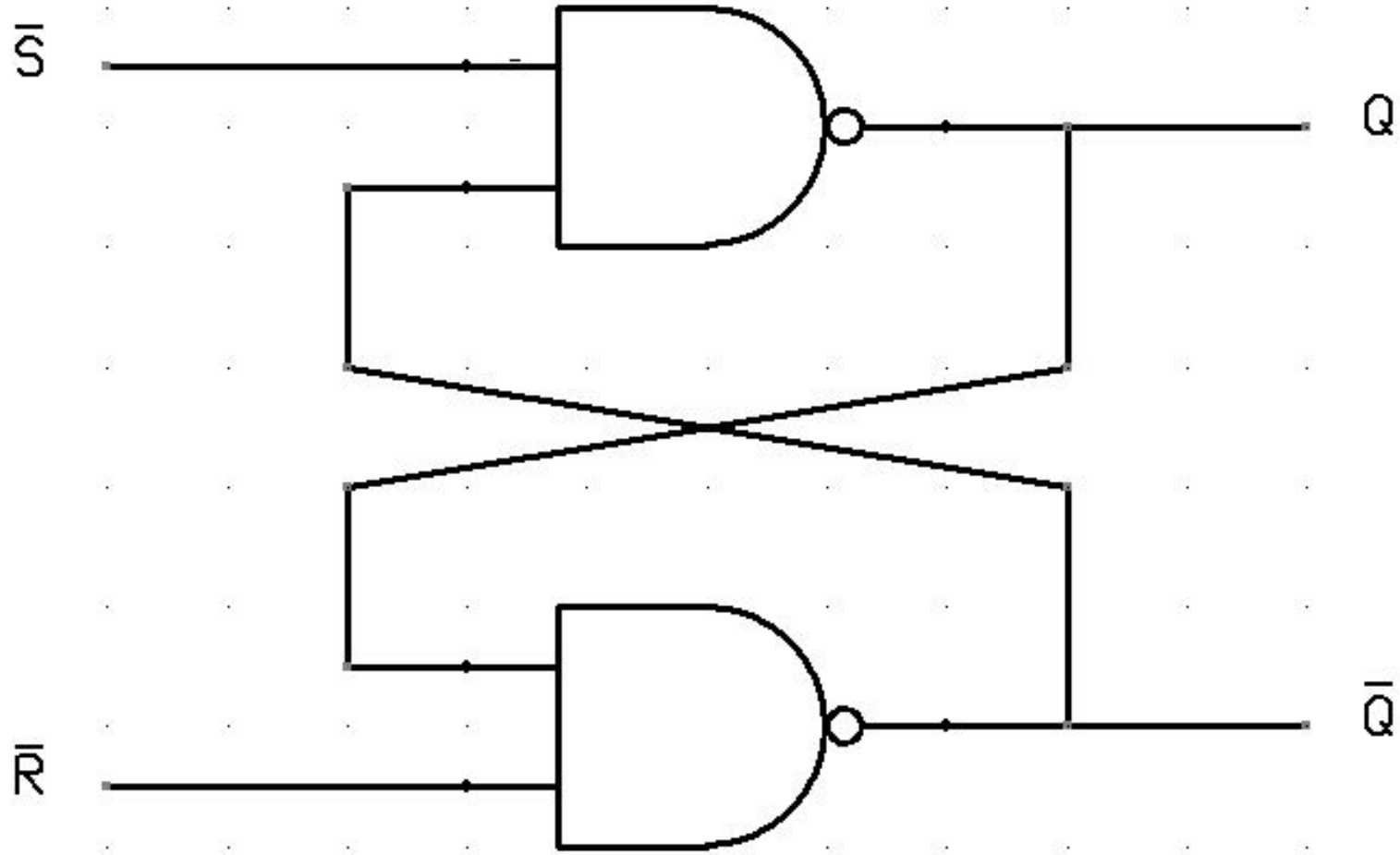


NOT

- 3 deęişkenli doęruluk tablosu((a AND b) OR c)

Durum No	Giriş A	Giriş B	Giriş C	1. Adım: (A VE B)	Sonuç: (A VE B) VEYA C
1	0	0	0	0	0
2	0	0	1	0	1
3	0	1	0	0	0
4	0	1	1	0	1
5	1	0	0	0	0
6	1	0	1	0	1
7	1	1	0	1	1
8	1	1	1	1	1

- Flip-Flop devresi diyagramı



Kaynakça

- http://www.edwardbosworth.com/CPSC2105/Lectures/Slides_05/Chapter_07/BusAndDisks_files/image002.jpg (Sayfa 8 görsel)
- <https://feelthebrain.files.wordpress.com/2017/04/978.jpg?w=730> (John von Neumann ve EDVAC fotoğrafı)
- http://hwitc95486cs.weebly.com/uploads/3/8/8/6/38863495/9226614_orig.jpg (Mantık kapıları doğruluk tabloları)
- <https://media.geeksforgeeks.org/wp-content/uploads/20240328125616/NOT-Gate.png> (Mantık kapıları doğruluk tabloları2)
- https://instrumentationtools.com/wp-content/uploads/2017/07/instrumentationtools.com_digital-logic-gates-truth-tables.png (Mantık kapısı sembolleri)
- <https://www.kasuo.com/wp-content/uploads/2025/05/Logic-Gates-Symbols-and-Truth-Tables.jpg> (Mantık kapısı sembolleri2)
- <https://electronicsforu.com/wp-content/uploads/2017/08/SR-latch.jpg> (flip-flop diyagramı)
- http://teach-ict.com/2016/images/diagrams/machine_code_structure.jpg (op-code operand fotoğrafı)
- <https://www.startpage.com/av/proxy-image?piurl=https%3A%2F%2Fimage1.slideserve.com%2F2747887%2Fexamples-of-opcodes-l.jpg&sp=1766523943T54b91f2b2f9a07299c33bc28fa111c9d97e28e3fa8a73260b54835554b60ed99> (op-code tablo)